

BURSA TEKNİK ÜNİVERSİTESİ

Mühendislik ve Doğa Bilimleri Fakültesi

Bilgisayar Mühendisliği Bölümü

BLM0324 — YAZILIM MÜHENDİSLİĞİ

Proje Ödevi

Uygulama Mimarisi Dökümanı

«Pick A Bite» — Yapay Zekâ Destekli Menü Öneri Sistemi

Öğrenci Numarası	Ad Soyad
22360859035	Tuğba Nur AYIK
22360859032	Muhammet Fatih GÖRAL
22360859047	Kubilay İNANÇ
22360859401	Suhail KHALEQI
22360859037	Ali YILMAZ
23360859019	Melike Rana YOZGATLI
23360859716	Raul NAMAZZADA
23360859736	Kübra KAYA
23360859402	Imer IMERI

Grup 8 · Mayıs 2026

İÇİNDEKİLER

1. Giriş	3
2. Mimari Genel Bakış	4
3. Sunum Katmanı (Mobil İstemci)	5
4. Backend Katmanı (Spring Boot)	6
5. Veri Katmanı	7
6. Yapay Zekâ Katmanı	8
7. Veri Toplama ve Harici Servisler	9
8. Güvenlik Mimarisi	9
9. Veri Akış Senaryoları	10
10. Performans ve Ölçeklenebilirlik	11
11. Teknoloji Yığını Özeti	11
12. Sonuç	11

1. GİRİŞ

Pick A Bite, kullanıcıların restoran menülerini daha hızlı, bilinçli ve kişisel ihtiyaçlarına uygun şekilde değerlendirmelerini sağlayan yapay zekâ destekli, konum tabanlı bir yemek öneri ve menü karşılaştırma sistemidir. Sistem; QR kod ile menü erişimi, harita üzerinden restoran keşfi, doğal dil işleyen chatbot ile öneri üretimi ve yakın çevredeki restoranları karşılaştırma özelliklerini bir araya getirir.

Bu doküman, Pick A Bite sisteminin uygulama mimarisini teknik ekibe, test ekibine ve proje paydaşlarına sunmak amacıyla hazırlanmıştır. Doküman; sistemin katmanlı yapısını, kullanılan teknolojileri, modül sorumluluklarını, veri akışlarını ve güvenlik tasarımını ayrıntılı olarak tanımlamaktadır.

1.1 Hedef Kitle

- Yazılım geliştirme ekibi (frontend, backend, AI geliştiricileri)
- Sistem tasarım ve mimari ekibi
- Test ve kalite güvence ekibi
- Proje danışmanları ve paydaşları

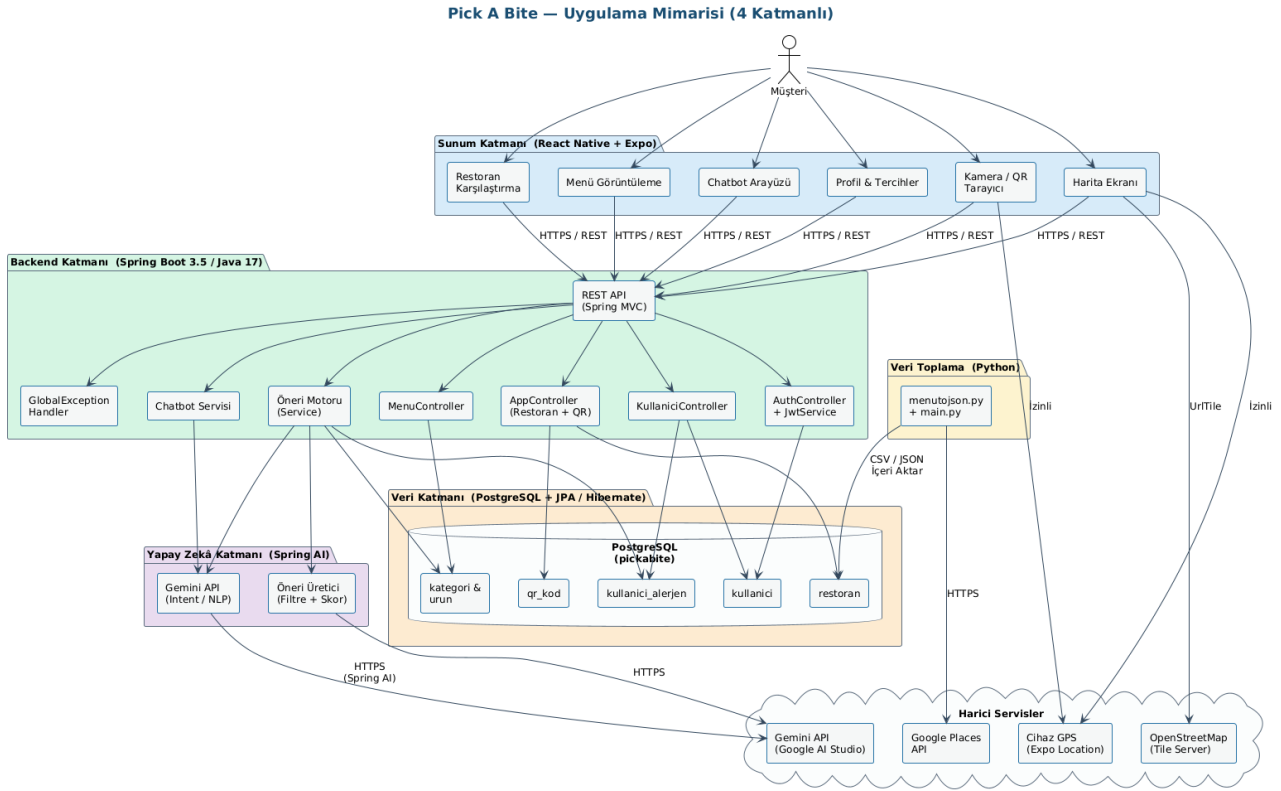
1.2 Kapsam

Doküman; sunum, backend, veri ve yapay zekâ katmanlarını, bu katmanlar arasındaki iletişim protokollerini, harici servis entegrasyonlarını ve uygulamanın uçtan uca veri akışını kapsar. Operasyonel detaylar (CI/CD, deployment ortamları) bu sürümün dışında tutulmuştur.

2. MİMARİ GENEL BAKIŞ

Pick A Bite mimarisi, katmanlı mimari (Layered Architecture) yaklaşımını benimser. Sistem dört temel katmandan oluşur: Sunum Katmanı, Backend Katmanı, Veri Katmanı ve Yapay Zekâ / İşleme Katmanı. Katmanlar arası iletişim tanımlı arayüzler üzerinden sağlanır; her katman kendi sorumluluk alanında izole çalışır.

Aşağıdaki bileşen diyagramı, sistemin kuş bakışı görünümünü ve katmanlar arasındaki iletişim akışını göstermektedir:



Şekil 2.1 — Pick A Bite 4 Katmanlı Uygulama Mimarisi

2.1 Katmanlar Arası İletişim

- Mobil istemci, backend ile HTTPS üzerinden REST API (Spring MVC) aracılığıyla iletişim kurar. Tüm uçlar /pick-a-bite/ prefix'i altında konumlanır.
- Backend, kalıcı veri için PostgreSQL veritabanına Spring Data JPA + Hibernate ile bağlanır.
- Backend, doğal dil işleme ve öneri üretimi için yapay zekâ katmanını çağırır; bu katman Spring AI 1.1.5 çerçevesi üzerinden Gemini API'ye HTTPS ile erişir.
- Mobil istemci, harita gösterimi için OpenStreetMap tile sunucusunu (react-native-maps + UrlTile) ve konum için cihaz GPS servisini (Expo Location) kullanır.
- Restoran verisi, ayrı bir Python script'i (Google Places API) tarafından dış kaynaklardan toplanır ve PostgreSQL'e içeri aktarılır.

3. SUNUM KATMANI (Presentation Layer)

Sunum katmanı, kullanıcının sistemle etkileşime girdiği mobil uygulama arayüzüdür. Tek kod tabanından hem Android hem iOS platformlarını desteklemek amacıyla React Native + Expo teknolojisi tercih edilmiştir. Dil olarak TypeScript kullanılarak tür güvenliği sağlanır.

3.1 Kullanılan Teknolojiler

Bileşen	Teknoloji / Kütüphane
Cross-platform framework	React Native 0.81 + Expo SDK
Dil	TypeScript 5.9
Navigasyon	Expo Router (file-based)
Harita	react-native-maps + OpenStreetMap (UrlTile)
Kamera & QR	expo-camera
Konum	expo-location (cihaz GPS)
Animasyon	react-native-reanimated
Yerel state	React Context / Zustand
HTTP istemci	fetch + interceptor (JWT header)

3.2 Ana Ekranlar

- Hesap Oluşturma / Giriş Ekranı: Yeni kullanıcı kaydı veya mevcut hesapla giriş; e-posta + şifre tabanlı.
- Beslenme Tercihleri Ekranı: Vegan, vejetaryen, glütensiz, helal, laktozsuz seçenekleri + alerjen listesi + bütçe limiti.
- Harita Ekranı (index.tsx): Kullanıcının bulunduğu konum etrafındaki restoranları MapView + OpenStreetMap tile'ları üzerinde pin olarak gösterir; arama çubuğu, QR tarama ve chatbot butonları içerir.
- Kamera / QR Ekranı (camera.tsx): Expo Camera ile QR kod tarar; geçerli kodlar menü sayfasına yönlendirir, geçersiz olanlar için uyarı mesajı gösterir.
- Menü Görüntüleme Ekranı: Restoran menüsünü kategorilere ayrılmış şekilde listeler; her ürün için fiyat, kalori ve alerjen bilgisi gösterir.
- Chatbot / Öneri Ekranı: Kullanıcının doğal dilde mesaj yazıp Gemini destekli öneri aldığı sohbet arayüzüdür.
- Restoran Detay Ekranı: Yakın çevredeki birden çok restoranın ürün, fiyat ve mesafe bilgilerini karşılaştırmalı sunar.

3.3 Kullanıcı İzinleri

- Konum izni: Restoran keşfi için gereklidir; reddedildiğinde liste görünümü sunulur.
- Kamera izni: QR kod okuma için gereklidir; reddedildiğinde kullanıcıya açıklayıcı uyarı gösterilir.

4. BACKEND KATMANI (Spring Boot)

Backend katmanı sistemin iş mantığı çekirdeğidir. Tüm mobil istemci istekleri önce Spring MVC dispatcher'a ulaşır, ardından ilgili controller tarafından işlenir ve gerektiğinde veri katmanı ile yapay zekâ katmanı ile iletişime geçilir. Backend, Spring Boot 3.5.14 (Java 17) üzerine RESTful prensiplere uygun şekilde inşa edilmiştir. Tüm uçlar /pick-a-bite/ prefix'i altında konumlanır.

4.1 Controller Yapısı

Controller	Sorumluluk	Örnek Endpoint
AuthController	Kayıt, giriş, kullanıcı kimliği — JWT üretim	POST /pick-a-bite/auth/giris
KullaniciController	Profil & beslenme/alerjen/bütçe tercihleri	PUT /pick-a-bite/kullanici/tercihler
AppController	Restoran CRUD, yakın restoran, QR doğrulama	GET /pick-a-bite/restoranlar/yakin
MenuController	Menü, kategori, ürün yönetimi	GET /pick-a-bite/restoranlar/{id}/menu
(Service) OneriMotoru	Tercih + bütçe + alerjen filtrelemesi	POST /pick-a-bite/oneri
(Service) Chatbot	Doğal dil sorgu — Gemini köprüsü	POST /pick-a-bite/chat/mesaj

4.2 Spring Security Filtreleri

- JwtAuthFilter: Korunmalı endpoint'lere yapılan her istek Authorization header'ı (Bearer JWT) içermelidir; jjwt 0.12.6 kütüphanesi ile imza doğrulanır.
- BCryptPasswordEncoder: Kullanıcı şifreleri kayıt sırasında BCrypt ile hash'lenir.
- CustomUserDetailsService: Spring Security için kullanıcı yükleme servisi.
- RestAuthenticationEntryPoint: Kimliksiz istekleri 401 JSON cevabı olarak yönlendirir.
- GlobalExceptionHandler (@RestControllerAdvice): Merkezî hata yakalama; kullanıcıya teknik olmayan dilde uygun mesajlar (BusinessException, ResourceNotFoundException) döndürür.

Not: Geliştirme aşamasında security konfigürasyonu SecurityConfigYedek sınıfında yorum satırı olarak hazır tutulmaktadır; nihai sürümde aktifleştirilecektir.

4.3 REST API Tasarım Prensipleri

- Kaynak odaklı URL'ler: /pick-a-bite/restoranlar, /pick-a-bite/restoranlar/{id}/menu
- Standart HTTP metodları: GET (oku), POST (oluştur), PUT (güncelle), DELETE (sil)
- JSON gövde: Tüm istek ve cevap gövdeleri UTF-8 JSON formatındadır; DTO sınıfları (DtoKullanici, DtoMenu, DtoRestoran, ...) ile şekillendirilir.
- Anlamlı HTTP statü kodları: 200, 201, 400, 401, 403, 404, 500
- JPA + Hibernate: Entity sınıfları (Kullanici, Restoran, Kategori, Urun) üzerinden veritabanı erişimi; ddl-auto = update ile şema otomatik senkronize edilir.

5. VERİ KATMANI

Veri katmanı, sistemin tüm kalıcı verilerini barındıran katmandır. İlişkisel veri modeli, oluşan ilişki yoğunluğu ve sorgu çeşitliliği nedeniyle PostgreSQL tercih edilmiştir. Backend, veritabanına Spring Data JPA + Hibernate ORM çerçevesi üzerinden bağlanır; şema, ddl-auto = update modunda entity sınıflarından otomatik üretilir. Veritabanı adı: pickabite.

5.1 Ana Tablolar

Tablo (JPA Entity)	Açıklama
kullanici	Kullanıcı hesabı + beslenme tercihleri (vegan, glütensiz, vb.) + bütçe
kullanici_alerjen	Kullanıcının alerjen listesi (@ElementCollection ile ayrı tablo)
restoran	Restoran kayıtları (ad, adres, telefon, çalışma saatleri, konum)
kategori	Menü kategorileri (her restoran için ayrı kayıtlar)
urun	Tek tek ürünler (ad, fiyat, açıklama, tahmini kalori, görsel URL)
qr_kod	Restoran QR kod kayıtları (sistem tarafından otomatik üretilir)

5.2 İndeks ve Performans Notları

- kullanici(email) üzerinde UNIQUE index — giriş sorguları için hızlı erişim.
- urun(kategori_id) üzerinde foreign-key index — menü listeleme performansı.
- qr_kod(kod) üzerinde UNIQUE index — QR doğrulama O(1).
- Konum bazlı sorgular için restoran(latitude, longitude) üzerinde composite index — Haversine mesafe hesabıyla yakın restoran arama.

5.3 Veri Güncellik Stratejisi

Restoran ve menü verileri iki kaynaktan beslenir:

- İlk yükleme (Python script): backend/menu/main.py Google Places API ile Bursa civarındaki restoranları çeker, Haversine mesafe filtresi uygulayarak restoranlar.csv ve menu.json dosyalarını üretir; bu veriler PostgreSQL'e içeri aktarılır.
- API üzerinden değişiklik: Menü, kategori ve ürün değişiklikleri MenuController üzerinden anlık olarak yansır.
- Hata durumları: Eksik veri durumlarında kullanıcıya «Menü bilgilerine şu anda erişilemiyor.» şeklinde anlaşılır bir mesaj gösterilir.

6. YAPAY ZEKÂ KATMANI

Pick A Bite'ın farklılığı yapay zekâ destekli öneri ve doğal dil etkileşimi özelliğinden gelir. Bu katman iki temel sorumluluk üstlenir: niyet sınıflandırması (intent analizi) ve kullanıcıya özel öneri üretimi. Entegrasyon, Spring AI 1.1.5 çerçevesi üzerinden Gemini API (Google AI Studio) ile sağlanır.

6.1 NLP Servisi (Intent Analizi) — Gemini API

Kullanıcı «250 TL bütçem var, glütensiz ve yüksek proteinli bir yemek istiyorum» gibi doğal dilde bir istek gönderdiğinde, Spring AI üzerinden Gemini'ye iletilen prompt şu yapılandırılmış elemanları çıkarır:

- Bütçe limiti: 250 TL
- Beslenme kısıtlamaları: glütensiz
- Besin odağı: yüksek proteinli
- Yemek tipi: ana yemek (çıkarsanır)

Bu yapılandırılmış çıktı, Öneri Motoru'na geçer ve menü üzerinde filtreleme + skorelama yapılır.

6.2 Öneri Motoru Algoritması

Öneri üretiminde takip edilen adımlar:

- 1. Katı Filtreler: Kullanıcı alerjenleri ile çelişen ürünler menüden çıkarılır.
- 2. Bütçe Filtresi: Belirtilen bütçe üstündeki ürünler elenir.
- 3. Tercih Skorelaması: Beslenme tercihi (vegan, glütensiz, vb.) ile uyumlu ürünlere ek puan verilir.
- 4. Niyet Eşleştirme: Kullanıcının sözel ifadesindeki anahtar kelimeler (tavuklu, hafif, vb.) ile ürün açıklaması Gemini ile semantik karşılaştırılır.
- 5. Sıralama: Skoru en yüksek 3-5 ürün kullanıcıya sunulur.
- 6. Boş Sonuç Yönetimi: Hiçbir ürün uygun değilse kullanıcıya neden bulunamadığı açıklanır ve kriter önerisi verilir.

6.3 Chatbot Sohbet Akışı

Kullanıcının chatbot ekranında girdiği mesajlar, Spring Boot tarafındaki Chatbot Servisi'ne iletilir. Servis, kullanıcının profil bilgilerini (tercih, alerjen, bütçe) ve mevcut menüyü Gemini'ye bağlam (context) olarak sunar; cevap olarak hem doğal dilde kısa bir açıklama hem de yapılandırılmış öneri listesi döner. Sohbet geçmişi, kullanıcı isteğiyle saklanabilir.

7. VERİ TOPLAMA VE HARİCİ SERVİSLER

Sistem, kalıcı veri için iç PostgreSQL veritabanı dışında dört harici servisten faydalanır. Restoran verisi geliştirme aşamasında ayrı bir Python script ile toplanır; AI işlemleri Gemini üzerinden Spring AI ile yönetilir; harita ise OpenStreetMap tile sunucusuna doğrudan bağlanır.

Servis	Sağlayıcı	Kullanım Amacı
Harita	OpenStreetMap (Tile Server)	Restoran pin gösterimi, harita gezinti
Konum	Cihaz GPS (Expo Location)	Kullanıcının anlık koordinatı
AI / NLP	Gemini API (Google AI Studio)	Intent analizi, öneri üretimi, sohbet
Veri Toplama	Google Places API + Python scripti	Restoran listesi ve menü ön verisi

8. GÜVENLİK MİMARİSİ

8.1 Kimlik Doğrulama

- JWT (jjwt 0.12.6): Giriş sonrası backend, kullanıcıya 24 saatlik (86 400 000 ms) bir access token üretir; istemci sonraki isteklerini Authorization: Bearer <token> başlığıyla yapar.
- BCryptPasswordEncoder: Şifreler veritabanında Spring Security'nin BCrypt encoder'ı ile hash'lenmiş olarak (sifreHash kolonu) saklanır.
- JwtAuthFilter: Her istek Spring Security filtre zincirinden geçer; geçerli JWT yoksa korumalı endpoint'e erişilemez.
- CustomUserDetailsService: Veritabanından kullanıcıyı yükleyip Spring Security context'ine yerleştirir.

Not: Şu an SecurityConfig sınıfı geliştirme kolaylığı için yorum satırı olarak hazır (SecurityConfigYedek); nihai sürümde aktif edilecektir.

8.2 Veri İletişim Güvenliği

- Tüm istemci-sunucu iletişimi HTTPS / TLS 1.3 üzerinden sağlanır.
- Hassas alanlar (şifre, JWT, Gemini API anahtarı) hiçbir log dosyasında yer almaz.
- Gemini API anahtarı application.properties içinde değil, ortam değişkeni veya secret manager üzerinden okunur.

8.3 Veri Gizliliği ve KVKK Uyumu

- Kullanıcı verileri yalnızca açık rıza ile işlenir.
- Konum verisi yalnızca kullanıcı izni ile alınır; sunucuda kalıcı olarak saklanmaz (yalnızca anlık sorgular için kullanılır).
- Kullanıcının profil silme talebi 'hesap silme' endpoint'i ile karşılanır; ilgili veriler 30 gün içinde tamamen silinir.

9. VERİ AKIŞ SENARYOLARI

Aşağıda sistemin uçtan uca çalışmasını gösteren üç tipik senaryo verilmiştir. Her senaryo ilgili tüm katmanları kapsar.

9.1 Senaryo 1 — QR Kod ile Menü Erişimi

- Müşteri kamera ile QR kodu okutur (Sunum Katmanı).
- Mobil istemci, GET /pick-a-bite/restoranlar/qr/{kod} endpoint'ine istek gönderir.
- Backend ApplicationController, qr_kod tablosunda kaydı sorgular (Veri Katmanı).
- Kod geçerliyse restoran ID döndürülür; istemci /pick-a-bite/restoranlar/{id}/menu çağırır.
- Backend MenuController menü verisini Hibernate ile çekip JSON olarak döndürür; istemci kategorize ekranda gösterir.
- Kod geçersizse GlobalExceptionHandler devreye girer ve kullanıcıya 'Bu QR kod sistemde tanımlı değil.' mesajı gösterilir.

9.2 Senaryo 2 — Chatbot ile Öneri Alma

- Müşteri 'Chatbot' ekranına mesaj yazar: «200 TL altında glütensiz bir yemek öner.»
- İstemci, mesajı POST /pick-a-bite/chat/mesaj üzerinden gönderir.
- Backend Chatbot Servisi, mesajı Spring AI üzerinden Gemini API'ye iletir.
- Gemini intent çıkarır: bütçe=200, kısıtlama=glütensiz, tip=yemek.
- Öneri Motoru, kullanıcının tercihleri ve mevcut menü üzerinden filtreleme + skortlama yapar.
- Uygun ürünler, fiyat ve açıklama ile birlikte istemciye döndürülür.
- Hiçbir ürün uymuyorsa kullanıcıya kriterleri esnetmesi önerilir.

9.3 Senaryo 3 — Harita Üzerinde Restoran Keşfi

- İstemci uygulama açılırken konum iznini ister (expo-location).
- İzin verildiğinde GPS koordinatı alınır.
- Mobil istemci, GET /pick-a-bite/restoranlar/yakin?enlem=&boylam=&radius= çağırır.
- Backend ApplicationController, Haversine mesafe hesabıyla yarıçap içindeki restoranları sorgular.
- Sonuçlar harita üzerinde (OpenStreetMap karoları + react-native-maps pin'leri ile) gösterilir; kullanıcının tercihleriyle uyumlu olanlar farklı renkte vurgulanır.

10. PERFORMANS VE ÖLÇEKLENEBİLİRLİK

10.1 Hedef Metrikler

- API yanıt süresi normal kullanımda < 2 saniye.
- Harita ve restoran listesi sorguları < 1 saniye.
- Chatbot yanıt süresi (AI dahil) < 4 saniye.
- Uygulama açılış süresi < 3 saniye.

10.2 Ölçekleme Stratejileri

- Yatay ölçekleme: Spring Boot uygulaması JWT ile stateless tasarlandı; load balancer arkasında birden fazla instance çalıştırılabilir.
- Veritabanı indeksleme: Konum, e-posta ve QR kod alanlarında kritik indeksler bulunur.
- Hibernate ikinci seviye cache: Sık erişilen entity'ler için JPA cache yapılandırılabilir (gelecek sürüm).
- CDN: Menü ürün görselleri CDN üzerinden sunulur (gelecek sürüm).

11. TEKNOLOJİ YIĞINI ÖZETİ

Katman	Teknoloji
Mobil istemci	React Native, Expo, TypeScript, Expo Router, react-native-maps
Backend	Spring Boot 3.5.14, Java 17, Spring MVC, Spring Data JPA
Auth & Security	Spring Security, jjwt 0.12.6, BCryptPasswordEncoder
Veritabanı	PostgreSQL + Hibernate ORM
Yapay Zekâ	Spring AI 1.1.5 + Gemini API
Harita	OpenStreetMap (react-native-maps + UrlTile)
Konum	Cihaz GPS (expo-location)
Veri Toplama	Python (pandas, requests) + Google Places API
Build & Geliştirme	Maven, Lombok, Spring Boot DevTools

12. SONUÇ

Pick A Bite, dört katmanlı modüler mimarisi sayesinde hem genişletilebilir hem de bakım kolaylığı yüksek bir yazılım altyapısına sahiptir. React Native ile her iki mobil platforma erişim, Spring Boot ile sağlam backend, PostgreSQL ile güvenilir veri kalıcılığı ve Spring AI + Gemini ile doğal dil odaklı kullanıcı deneyimi sunulur.

Bu mimari, gereksinim dokümanında tanımlanan tüm işlevsel gereksinimleri karşılar. Gelecek sürümlerde ödeme entegrasyonu, sosyal yorum ve sağlık servisi gibi özellikler eklenebilir.